

SEMESTER PROJECT REPORT

Web Programming (MERN Stack)

Event Management System (EMS)

NexusEvents — Your Premier Event Platform

Roll No.	Student Name	Role
F230822	Nehza	Frontend Developer
F230829	Khadija Younas	Backend Developer
F230573	Ghulam Dastageer	Database & API Integration
F230517	Taha Saleem	Full Stack & Documentation

Submitted To:

Sir Saqib Ameer

Department of Computer Science

Session: 2023–2027 | Date: May 2026

Table of Contents

No.	Section	Page
1	Introduction & Project Overview	3
2	Problem Statement	3
3	Objectives	4
4	Project Scope	4
5	Methodology	5
6	System Architecture	5
7	Technology Stack	6
8	Database Design	7
9	Backend Implementation	8
10	Frontend Implementation	9
11	API Endpoints	10
12	Features Implemented	11
13	Challenges & Solutions	11
14	Testing	12
15	Source Code	13
16	Conclusion & Future Work	20

1. Introduction & Project Overview

The Event Management System (EMS), branded as NexusEvents, is a full-stack web application developed as part of the Web Programming semester project. It serves as a centralized digital platform that streamlines the complete lifecycle of event management — from creation and publication to discovery by attendees.

In today's fast-paced world, organizing events manually through spreadsheets, emails, or paper-based methods is inefficient and error-prone. NexusEvents bridges this gap by providing a modern, responsive, and intuitive platform where event organizers can publish their events and participants can browse and discover them with ease.

The system is built on the MERN (MongoDB, Express.js, React, Node.js) stack — a widely adopted technology combination in modern web development — ensuring high performance, scalability, and maintainability.

2. Problem Statement

Traditional event management faces several key challenges that hinder efficiency and user experience:

- No centralized platform: Organizers use scattered tools (WhatsApp groups, posters, emails) with no unified system.
- Poor discoverability: Participants struggle to find relevant events in their area without a dedicated platform.
- Manual data management: Event details (venue, time, pricing) are managed manually, leading to errors and outdated information.
- Lack of real-time updates: Changes to event details are not instantly propagated to all interested participants.
- No structured data storage: Without a proper database, event history, analytics, and management become nearly impossible.

NexusEvents solves all these problems by providing a RESTful API-powered backend connected to a MongoDB database, and a clean React-based frontend that delivers a seamless user experience.

3. Objectives

The primary objectives of this project are:

1. Develop a full-stack MERN application that demonstrates real-world software development practices.
2. Implement a RESTful API for complete CRUD operations on event data.
3. Design a responsive and visually appealing React frontend with component-based architecture.
4. Integrate MongoDB as the persistent data store with a well-defined Mongoose schema.
5. Enable event organizers to publish events with rich details including title, description, date, time, location, organizer name, ticket price, and cover image.
6. Enable participants to browse all upcoming events and view detailed information for each.
7. Demonstrate proper project structure, modular code organization, and industry-standard conventions.

4. Project Scope

4.1 In Scope

- Event creation with full metadata (title, description, date, time, location, organizer, price, image)
- Event listing page showing all upcoming events in a card-based grid layout
- Event detail page with full event information
- RESTful API with GET, POST, PUT, DELETE endpoints
- MongoDB database integration via Mongoose ODM
- Responsive design using React-Bootstrap
- Client-side routing with React Router DOM
- CORS-enabled API for cross-origin requests

4.2 Out of Scope (Future Work)

- User authentication and role-based access control
- Ticket booking and payment gateway integration
- Email notifications and reminders
- Event ratings and reviews
- Admin dashboard with analytics

5. Methodology

The project followed an Agile-inspired iterative development approach, divided into the following phases:

Phase	Activities	Duration
Phase 1 — Planning	Requirements analysis, technology selection, task distribution among team members	Week 1
Phase 2 — Backend Dev	Set up Express server, define Mongoose schema, implement controllers and routes	Week 2
Phase 3 — Frontend Dev	Bootstrap React app, create components (Navbar, Footer, Home, CreateEvent, EventDetails)	Week 3
Phase 4 — Integration	Connect frontend to backend via Axios HTTP calls, test all API endpoints	Week 4
Phase 5 — Testing & Docs	Manual testing, bug fixing, report writing, viva preparation	Week 5

Team collaboration was managed through task division based on each member's strengths. Regular team meetings were held to review progress and resolve integration issues.

6. System Architecture

The application follows a classic 3-tier client-server architecture:

Tier	Technology	Responsibility
Presentation Tier	React.js + React-Bootstrap	Renders the UI, handles user interactions, makes HTTP requests
Application Tier	Node.js + Express.js	Processes requests, applies business logic, routes API calls
Data Tier	MongoDB + Mongoose ODM	Persists event data, enforces schema validation

Data Flow: The React frontend sends HTTP requests (via Axios) to the Express.js REST API running on Node.js. The API processes requests, interacts with MongoDB using Mongoose, and returns JSON responses. The frontend then renders the data dynamically.

7. Technology Stack

Category	Technology	Version	Purpose
Runtime	Node.js	LTS	JavaScript server-side runtime
Framework	Express.js	5.2.x	REST API framework
Database	MongoDB	Latest	NoSQL document database
ODM	Mongoose	9.6.x	Schema definition & DB interaction
Frontend	React	19.x	UI component library
Routing	React Router DOM	7.x	Client-side navigation
HTTP Client	Axios	1.x	API calls from frontend
UI Library	React-Bootstrap	2.x	Responsive UI components
Icons	React Icons	5.x	SVG icon library
Environment	dotenv	17.x	Environment variable management
CORS	cors	2.8.x	Cross-origin resource sharing

8. Database Design

8.1 Database: MongoDB

MongoDB is used as the primary database due to its flexible document-oriented storage model, which suits the dynamic nature of event data. The database is named 'ems' (Event Management System).

8.2 Collection: events

The application uses a single collection named 'events'. Each document follows the schema defined in the Mongoose model:

Field	Type	Required	Validation / Default
title	String	Yes	Max 100 characters, trimmed
description	String	Yes	Free text, no length limit
date	Date	Yes	Valid JavaScript Date object
time	String	Yes	Stored as HH:MM string
location	String	Yes	Venue name or address
imageUrl	String	No	Defaults to a pre-set Unsplash URL
organizer	String	Yes	Name of the organizing entity
price	Number	No	Defaults to 0 (free event)
createdAt	Date	No	Auto-set to Date.now()

Mongoose automatically creates an `_id` field (ObjectId) as the primary key for each document. Events are sorted by date in ascending order when fetched.

9. Backend Implementation

9.1 Project Structure

```
backend/
├── config/
│   └── db.js           # MongoDB connection logic
├── controllers/
│   └── eventController.js # Business logic for all event operations
├── models/
│   └── Event.js        # Mongoose schema & model
├── routes/
│   └── eventRoutes.js   # Route definitions
├── .env                # Environment variables (PORT, MONGO_URI)
└── server.js           # App entry point
```

9.2 Server Configuration (server.js)

The Express application is configured with the following middleware chain:

- `cors()` — Allows cross-origin requests from the React development server
- `express.json()` — Parses incoming JSON request bodies
- Route mounting: All event-related routes are mounted at `/api/events`
- Global error handler middleware for 500-level errors

9.3 Database Connection (config/db.js)

The `connectDB` function uses `async/await` to establish a Mongoose connection. If the connection fails, the process exits to prevent the server from running without a database. The connection URI is read from the `MONGO_URI` environment variable with a `localhost` fallback.

9.4 Event Controller (controllers/eventController.js)

Five controller functions handle all event operations:

Function	HTTP Method	Description
<code>getEvents</code>	GET	Fetches all events sorted by date ascending
<code>getEvent</code>	GET	Fetches a single event by MongoDB ObjectId
<code>createEvent</code>	POST	Creates a new event after Mongoose validation
<code>updateEvent</code>	PUT	Updates an existing event by id with validators
<code>deleteEvent</code>	DELETE	Soft-finds then hard-deletes an event by id

Each controller wraps logic in try/catch blocks. `createEvent` specifically checks for Mongoose `ValidationError` and returns readable error messages extracted from the error object.

10. Frontend Implementation

10.1 Project Structure

```
frontend/src/
├── components/
│   ├── NavbarComponent.js # Sticky top navigation
│   ├── Footer.js          # Site footer with links
│   └── EventCard.js        # Reusable event card component
├── pages/
│   ├── Home.js            # Landing page with hero + events grid
│   ├── CreateEvent.js     # Event creation form
│   └── EventDetails.js    # Full event detail view
├── App.js                 # Root component with Router & routes
└── index.js               # ReactDOM render entry point
```

10.2 Component Overview

Component	Type	Key Features
NavbarComponent	Layout	React-Bootstrap Navbar; sticky-top; React Router Links; brand logo with accent color
Footer	Layout	3-column layout; quick links; contact info; dynamic copyright year
Home	Page	Hero banner with parallax overlay; fetches all events via useEffect; responsive 3-col card grid
EventCard	UI Widget	Card with event image, title, date/location badges, price tag, 'View Details' link
CreateEvent	Page/Form	Controlled form with all Event fields; random default image if none provided; POST to API
EventDetails	Page	Fetches single event; full hero banner; detailed sidebar with icons; date/time/location/organizer

10.3 State Management

The application uses React's built-in `useState` and `useEffect` hooks for local state management. `Axios` is used for all HTTP requests. Loading and error states are managed at the component level to provide proper user feedback.

11. API Endpoints

Base URL: <http://localhost:5000/api>

Method	Endpoint	Description	Request Body	Success Response
GET	/events	Get all events (sorted by date)	None	200 + {success, count, data[]}
GET	/events/:id	Get a single event by ID	None	200 + {success, data}
POST	/events	Create a new event	JSON event obj	201 + {success, data}
PUT	/events/:id	Update an event by ID	JSON fields	200 + {success, data}
DELETE	/events/:id	Delete an event by ID	None	200 + {success, data: {}}

Error Responses: 404 {success: false, error: 'Event not found'} | 400 {success: false, error: [validation messages]} | 500 {success: false, error: 'Server Error'}

12. Features Implemented

Feature	Status	Description
Event Listing	Implemented	Home page shows all events in a responsive grid
Event Creation	Implemented	Full form to create events with validation
Event Details View	Implemented	Dedicated page with hero image and all details
REST API (CRUD)	Implemented	All 5 endpoints fully functional
MongoDB Persistence	Implemented	Data persists across sessions via MongoDB
Responsive Design	Implemented	Works on mobile, tablet, and desktop
Client-side Routing	Implemented	React Router DOM v7 for SPA navigation
Loading States	Implemented	Spinners shown during API calls
Error Handling	Implemented	User-friendly error messages on failures
Default Event Images	Implemented	Auto-assigned Unsplash images when none provided
Free/Paid Event Badges	Implemented	Price displayed as badge on event cards
Sticky Navigation	Implemented	Navbar stays at top on scroll

13. Challenges & Solutions

Challenge	Solution
CORS errors when frontend called the backend API	Installed and configured the 'cors' npm package in Express to allow all origins during development
Mongoose validation errors returning unclear messages	Extracted error messages using <code>Object.values(err.errors).map(val => val.message)</code> for user-friendly output
React Router v7 breaking changes from v6	Updated all route and link syntax to match React Router DOM v7 conventions
Default images needed when user provides no URL	Implemented a curated array of Unsplash image URLs; randomly assigned on form submit
Date formatting inconsistency between MongoDB and display	Used JavaScript's <code>toLocaleDateString()</code> on the date received from MongoDB to format for display
Express v5 body parser changes	Used <code>express.json()</code> directly (no need for separate body-parser package in Express v5)

14. Testing

14.1 Manual API Testing

All API endpoints were tested manually using Postman/Thunder Client:

Test Case	Method	Input	Expected	R e s u l t
Get all events	GET	No body	200 with events array	P A S S
Get event by valid ID	GET	Valid ObjectId	200 with event object	P A S S
Get event by invalid ID	GET	Non-existent ID	404 error message	P A S S
Create valid event	POST	All required fields	201 created event	P A S S
Create event missing title	POST	title omitted	400 validation error	P A S S
Update event	PUT	Valid ID + new data	200 updated event	P A S S
Delete event	DELETE	Valid ID	200 empty data	P A S S

14.2 Frontend Testing

- Navigation between Home, Create Event, and Event Details pages verified
- Form submission with all fields, partial fields, and missing required fields tested

- Spinner visibility during API loading verified
- Error alert display when backend is unreachable tested
- Responsive layout tested on Chrome DevTools at 375px (mobile), 768px (tablet), 1280px (desktop)

15. Source Code

Complete source code for all project files is listed below, organized by layer.

15.1 Backend — server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const eventRoutes = require('./routes/eventRoutes');

dotenv.config();
connectDB();

const app = express();

app.use(cors());
app.use(express.json());
app.use('/api/events', eventRoutes);

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ success: false, error: 'Server Error' });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

15.2 Backend — config/db.js

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(
      process.env.MONGO_URI || 'mongodb://localhost:27017/ems'
    );
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (err) {
    console.error(`Error: ${err.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

15.3 Backend — models/Event.js

```
const mongoose = require('mongoose');

const EventSchema = new mongoose.Schema({
  title: {
    type: String, required: [true, 'Please add a title'],
    trim: true, maxlength: [100, 'Title cannot exceed 100 chars']
  },
  description: { type: String, required: [true, 'Please add a description'] },
  date: { type: Date, required: [true, 'Please add a date'] },
  time: { type: String, required: [true, 'Please add a time'] },
  location: { type: String, required: [true, 'Please add a location'] },
  imageUrl: {
    type: String,
    default: 'https://images.unsplash.com/photo-1505236858219-...'
  },
  organizer: { type: String, required: [true, 'Please add an organizer'] },
  price: { type: Number, default: 0 },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Event', EventSchema);
```

15.4 Backend — routes/eventRoutes.js

```
const express = require('express');
const {
  getEvents, getEvent, createEvent, updateEvent, deleteEvent
} = require('../controllers/eventController');

const router = express.Router();

router.route('/').get(getEvents).post(createEvent);
router.route('/:id').get(getEvent).put(updateEvent).delete(deleteEvent);

module.exports = router;
```


15.5 Backend — controllers/eventController.js

```
const Event = require('../models/Event');

exports.getEvents = async (req, res, next) => {
  try {
    const events = await Event.find().sort({ date: 1 });
    res.status(200).json({ success: true, count: events.length, data: events });
  } catch (err) { res.status(500).json({ success: false, error: 'Server Error' }); }
};

exports.getEvent = async (req, res, next) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ success: false, error: 'Event not found' });
    res.status(200).json({ success: true, data: event });
  } catch (err) { res.status(500).json({ success: false, error: 'Server Error' }); }
};

exports.createEvent = async (req, res, next) => {
  try {
    const event = await Event.create(req.body);
    res.status(201).json({ success: true, data: event });
  } catch (err) {
    if (err.name === 'ValidationError') {
      const messages = Object.values(err.errors).map(v => v.message);
      return res.status(400).json({ success: false, error: messages });
    }
    res.status(500).json({ success: false, error: 'Server Error' });
  }
};

exports.updateEvent = async (req, res, next) => {
  try {
    const event = await Event.findByIdAndUpdate(req.params.id, req.body, {
      new: true, runValidators: true
    });
    if (!event) return res.status(404).json({ success: false, error: 'Event not found' });
    res.status(200).json({ success: true, data: event });
  } catch (err) { res.status(500).json({ success: false, error: 'Server Error' }); }
};

exports.deleteEvent = async (req, res, next) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ success: false, error: 'Event not found' });
    await event.deleteOne();
    res.status(200).json({ success: true, data: {} });
  } catch (err) { res.status(500).json({ success: false, error: 'Server Error' }); }
};
```

15.6 Frontend — NavbarComponent.js

```
import React from 'react';
import { Navbar, Nav, Container } from 'react-bootstrap';
import { Link } from 'react-router-dom';

const NavbarComponent = () => (
  <Navbar expand='lg' className='custom-navbar' variant='dark' sticky='top'>
    <Container>
      <Navbar.Brand as={Link} to='/'>
        <span style={{ color: 'var(--accent-color)' }}>Nexus</span>Events
      </Navbar.Brand>
      <Navbar.Toggle aria-controls='basic-navbar-nav' />
      <Navbar.Collapse id='basic-navbar-nav'>
        <Nav className='ms-auto'>
          <Nav.Link as={Link} to='/'>Home</Nav.Link>
          <Nav.Link as={Link} to='/create-event'>Create Event</Nav.Link>
        </Nav>
      </Navbar.Collapse>
    </Container>
  </Navbar>
);

export default NavbarComponent;
```

15.7 Frontend — Footer.js

```
import React from 'react';
import { Container, Row, Col } from 'react-bootstrap';

const Footer = () => (
  <footer className='custom-footer'>
    <Container>
      <Row>
        <Col md={4}>
          <h5><span style={{ color: 'var(--accent-color)' }}>Nexus</span>Events</h5>
          <p>Your premier platform for discovering extraordinary events.</p>
        </Col>
        <Col md={4}>
          <h5>Quick Links</h5>
          <ul className='list-unstyled'>
            <li><a href='/'>Home</a></li>
            <li><a href='/create-event'>Create Event</a></li>
          </ul>
        </Col>
        <Col md={4}>
          <h5>Contact Us</h5>
          <p>Email: info@nexusevents.com<br/>Phone: +1 (555) 123-4567</p>
        </Col>
      </Row>
      <div className='text-center mt-4'>
        <p>&copy; {new Date().getFullYear()} NexusEvents. All rights reserved.</p>
      </div>
    </Container>
  </footer>
);

export default Footer;
```

15.8 Frontend — pages/Home.js

```
import React, { useState, useEffect } from 'react';
import { Container, Row, Col, Spinner, Alert } from 'react-bootstrap';
import axios from 'axios';
import EventCard from '../components/EventCard';

const Home = () => {
  const [eventsList, setEventsList] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchEvents = async () => {
      try {
        const res = await axios.get('http://localhost:5000/api/events');
        setEventsList(res.data.data);
        setLoading(false);
      } catch (err) {
        setError('Failed to fetch events. Is the backend running?');
        setLoading(false);
      }
    };
    fetchEvents();
  }, []);

  return (
    <div>
      { /* Hero Section */ }
      <div style={{ backgroundImage: 'linear-gradient(..., url(...)', ... }}>
        <Container>
          <h1>Discover Extraordinary Events</h1>
          <p>Join the best conferences, meetups, and concerts.</p>
        </Container>
      </div>
      <Container className='my-5'>
        <h2>Upcoming Events</h2>
        {loading ? <Spinner /> : error ? <Alert>{error}</Alert> : (
          <Row xs={1} md={2} lg={3} className='g-4'>
            {eventsList.map(e => <Col key={e._id}><EventCard event={e} /></Col>)}
          </Row>
        )}
      </Container>
    </div>
  );
};

export default Home;
```

15.9 Frontend — pages/CreateEvent.js (Key Sections)

```
import React, { useState } from 'react';
import { Container, Row, Col, Form, Button, Card, Alert } from 'react-bootstrap';
import { useNavigate } from 'react-router-dom';
import axios from 'axios';

const CreateEvent = () => {
  const navigate = useNavigate();
  const [formData, setFormData] = useState({
    title: '', description: '', date: '', time: '',
    location: '', organizer: '', price: 0, imageUrl: ''
  });
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  const defaultImages = [
    'https://images.unsplash.com/photo-1540575467063-...', // Concert
    'https://images.unsplash.com/photo-1505373877841-...', // Tech
    'https://images.unsplash.com/photo-1511578314322-...', // Dinner
    'https://images.unsplash.com/photo-1523580494112-...', // Workshop
  ];

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true); setError(null);
    const data = { ...formData };
    if (!data.imageUrl)
      data.imageUrl = defaultImages[Math.floor(Math.random() * 4)];
    try {
      await axios.post('http://localhost:5000/api/events', data);
      navigate('/');
    } catch (err) {
      setError(err.response?.data?.error || 'Something went wrong');
    } finally { setLoading(false); }
  };
  // ... Form JSX with all input fields ...
};

export default CreateEvent;
```

15.10 Frontend — pages/EventDetails.js (Key Sections)

```
import React, { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import axios from 'axios';
import { FaCalendarAlt, FaMapMarkerAlt, FaUserTie } from 'react-icons/fa';

const EventDetails = () => {
  const { id } = useParams();
  const [event, setEvent] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

```
useEffect(() => {
  axios.get(`http://localhost:5000/api/events/${id}`)
    .then(res => { setEvent(res.data.data); setLoading(false); })
    .catch(() => { setError('Event not found. '); setLoading(false); });
}, [id]);

// Renders hero banner + two-column layout:
// Left: description card | Right: details sidebar with icons
// Date | Time | Location | Organizer | Price badge
};

export default EventDetails;
```

16. Conclusion & Future Work

16.1 Conclusion

The Event Management System (NexusEvents) was successfully developed as a complete MERN stack web application. The project demonstrates proficiency in all four core technologies of the stack — MongoDB for persistent storage, Express.js for RESTful API development, React for dynamic frontend interfaces, and Node.js as the runtime environment.

The team of four members collaborated effectively, dividing responsibilities across frontend, backend, database, and documentation domains. All core features were implemented and tested. The application is fully functional with live API integration between the React frontend and the Node.js/Express backend.

This project provided hands-on experience with real-world software development practices including modular code organization, REST API design, Mongoose schema validation, component-based UI development, and React Hooks for state management.

16.2 Future Enhancements

- User Authentication: JWT-based login/registration to differentiate organizers from attendees
- Ticket Booking: Allow users to register/book seats for events with capacity limits
- Payment Integration: Stripe or PayPal integration for paid event ticketing
- Search & Filter: Filter events by category, date range, location, and price
- Admin Dashboard: Analytics panel showing event statistics, registrations, and revenue
- Email Notifications: Automated confirmation emails and event reminders via Nodemailer
- Image Upload: Direct image upload via Cloudinary instead of requiring image URLs
- Reviews & Ratings: Post-event rating system for attendee feedback

— *End of Report* —

NexusEvents | Event Management System | MERN Stack | Web Programming
F230822 Nehza | F230829 Khadija Younas | F230573 Ghulam Dastageer | F230517 Taha Saleem